

Sessions Are Braids: Measuring Latent Thread Structure in Real Coding-Agent Sessions

Thamilvendhan Munirathinam

mthamil107@gmail.com · July 2026 · Measurement note and proposal

doi:10.5281/zenodo.21486184 · <https://github.com/mthamil107/sessions-are-braids>

ABSTRACT. Coding agents such as Claude Code accumulate conversation history in a single linear, append-only context window. We measure, on a corpus of real multi-turn Claude Code sessions from one professional user (68–70 sessions depending on measurement filter, §3; 4,434 user prompts, ≈ 17 million estimated stored-context tokens after de-duplicating forked-session re-logs), what that window is actually made of — first by turn *intent*, then by latent *topic structure*. Two findings. **(1) Negative:** pure-comprehension turns (“explain this”, “why?”) — the most safely evictable content, since the model can regenerate any explanation from what remains — account for only **2.3%** of stored tokens (median session 1.6%; heuristic upper bound). Intent-based eviction of explanation turns is not worth building for professional sessions. **(2) Positive:** sessions are braids. **93%** of sessions interleave topically distinct threads, returning to earlier threads after working on others; blind LLM re-segmentation of a 10-session stratified sample (30 independent passes, adjudicated by a two-judge panel) puts the cross-thread share of each prompt’s accumulated context at **54–63%** — higher than the lexical heuristic’s 25–45% estimate, whose switch points the validation rejected (29% precision). **(3) Component ablation:** replaying 18 real thread-return moments under three context arms — full, summary-only (–50% size), and summary+entity-ledger (–43%) — shows **entity retention is the necessary condensation ingredient**: it strictly dominates summary-only and exhibits the fewest dependency failures of all arms, fewer even than full context (10 vs 14), while full context retains a shrinking, non-significant preference edge (9–5, from 11–5 without entities). This quantifies, in real usage data, the “logical context poisoning” that the Conversation Tree Architecture recently formalized, and targets exactly the components that work leaves open: *automatic thread placement* and *condensation*. All numbers are from a single-user corpus; scripts and aggregate data are released.

1 Introduction

Every mainstream LLM agent interface — chat apps, coding agents, agent SDKs — represents a conversation as a linear, append-only message list. Everything the user and agent have said, and every tool result, accumulates in one shared context window until a compaction step summarizes it away. The recently proposed Conversation Tree Architecture (CTA) names the resulting failure mode *logical context poisoning*: topically distinct threads bleed into one another and progressively degrade response quality [1]. CTA formalizes conversations as trees whose nodes hold isolated local contexts, but its prototype is fully manual, and its authors list as open problems (a) deciding automatically where an incoming message belongs and (b) selectively condensing a finished branch back into its parent. Meanwhile Claude Code shipped `/branch` — manual copy-and-diverge session forking with tree-grouped session navigation [5] — confirming product demand for tree structure, but with no automatic placement and no merge-back.

Missing from both the research and the products is a basic empirical question: *how much tree structure is actually hiding inside real linear sessions?* If real sessions are essentially linear, routers and trees are machinery without a workload. If they interleave heavily, then linear history is the wrong data structure for agents, and every session silently pays a context-poisoning tax. This

note answers that question on real usage data, and reports a second measurement — a negative result on a related idea — for calibration.

Contributions:

- **An intent breakdown of real coding-agent context** (§4): by stored tokens, 83.8% of context is action exchanges (edits, commands, and their tool outputs); pure-comprehension explanation turns are 2.3%. A pre-registered go/no-go gate ($\geq 20\%$ proceed, $< 10\%$ stop) kills intent-based *eviction* of explanation turns as a mechanism. We report this negative result in full.
- **The braid measurement** (§5): 93% of substantive sessions interleave topically distinct threads (robust across segmentation thresholds), and 25–45% of the prior context each prompt carries belongs to other threads (“crossload”) — an upper bound on what ancestor-path-only context with merge-back condensation could save.
- **A proposal with an evaluation plan** (§6): an automatic thread router — classify each incoming prompt as *continuation*, *sibling*, or *return*; serve each thread only its ancestor path plus condensed conclusions of closed threads — and a replay-ablation design that measures whether auto-placed trees beat linear sessions on tokens *and* task quality.

2 Related work

Existing context-reduction mechanisms prune along three axes: **bulk** (large tool outputs; Claude Code microcompaction, Anthropic context editing [6]), **age** (compaction and summarization of old turns), and **similarity** (retrieval-style memory: MemGPT [7], vector-store chat memory). None classifies turns by what they were *for*, and none uses topic membership as the unit of context isolation. Recent agent-efficiency papers supply the motivation: pruning stale tool-interaction units improved task success from 71% to 91.6% while cutting tokens 1.48M→553K [2], and RL-trained context curation achieved up to 8× token reduction with higher success [3] — evidence that irrelevant context actively *hurts*, not merely costs. On structure, CTA [1] formalizes tree-structured conversations with context-isolated nodes; Branchat [4] and canvas-style interfaces [8] explore tree/spatial UIs for revisitation; Claude Code’s `/branch` [5] ships manual session forking. All placement in these systems is manual, condensation is absent or all-or-nothing, and none reports measurements of latent thread structure in real sessions. That measurement, and the automatic router it motivates, are this note’s subject.

3 Corpus and method

Claude Code stores complete session transcripts as JSONL event streams under `~/.claude/projects/`. Our corpus is the author’s complete history: $\approx 2,150$ transcript files (≈ 687 MB; the count grows as the author keeps working) across 38 project directories, of which $\approx 1,140$ are subagent sidechains and most of the rest are short one-question sessions. Events are grouped into *exchanges*: one real user prompt plus all assistant activity (text, tool calls, tool results) until the next real prompt. Harness-generated pseudo-prompts (tool results, background task notifications, interruption markers) are folded into the current exchange rather than counted as user intent. An exchange’s *stored-token* contribution is estimated as `chars/4` of its persisted content — what actually occupies the window — cross-checked against the transcripts’ native output-token usage fields. Forked/resumed sessions re-log their inherited history with duplicate event uuids; all measurements de-duplicate at the event level (keep first occurrence). Sessions with ≥ 3 substantive exchanges enter the intent measurement (70 sessions); sessions with ≥ 5 real prompts enter the thread measurement (68 sessions, 4,434 prompts). The corpus is one

professional user’s real work (software engineering and research sessions) — a deliberate first-person feasibility corpus, with the external-validity limits discussed in §7.

4 Measurement 1 (negative result): explanation turns are too small to be worth evicting

Hypothesis: comprehension turns — “explain this”, “why did you do that?”, “in plain English” — are *regenerable* (the model can re-derive any explanation from the code and decisions that remain), hence the safest content to evict at task boundaries; if they are a large token share, intent-based eviction is a mechanism worth building. Exchanges were labeled by heuristics: any state-changing tool call → ACTION; read-only tool calls only → RESEARCH; no tools plus decision language → DECISION; no tools plus explain-pattern prompt or long code-free prose reply → COMPREHENSION; else OTHER. The gate, fixed before measuring: $\geq 20\%$ of tokens evictable → build; $< 10\%$ → stop.

Label	Stored tokens	Share
ACTION (edits, commands, tool outputs)	13,919,743	83.8%
DECISION	1,001,042	6.0%
RESEARCH (read-only tool use)	760,629	4.6%
OTHER	552,758	3.3%
COMPREHENSION (evictable candidate)	382,036	2.3%

Table 1. Intent breakdown of stored context across 70 sessions ($\approx 16.6\text{M}$ estimated tokens, uuid-deduped). Per-session comprehension share: median 1.6%, maximum 17.4%; zero sessions reach 20%, one exceeds 10%. The heuristic over-counts comprehension (it is an upper bound: no retrospective reference check was applied).

The gate fires decisively: even the upper bound sits far below the stop threshold. Perfect eviction of every explanation turn would extend session headroom by $\approx 2\%$ — within measurement noise. The prediction that tool outputs dominate real agent context (70–90% in our pre-registered adversarial review) was confirmed and exceeded. We conclude that intent-based *eviction of explanation turns* is not a viable mechanism for professional coding sessions, and note that the learner/“vibe-coder” segment — plausibly much heavier in comprehension turns — cannot be assessed from this corpus. The taxonomy and the regenerability principle, however, survive; §5 applies them at a different granularity, where the effect is an order of magnitude larger.

5 Measurement 2: sessions are braids

We next segment each session’s real prompts into topical *threads*, using lexical similarity between a prompt’s content terms (plus files touched by tools in its exchange) and the accumulated term/file profile of each existing thread. Each prompt is labeled CONT (continues the current thread), NEW (opens a thread — a sibling branch), or RETURN (resumes an earlier thread after working on something else). Short reactive prompts (“ok”, “yes”, “continue”) are continuations by rule. Because lexical segmentation can over-split, we report two operating points: the default similarity threshold ($\theta=0.15$) and a deliberately generous merging threshold ($\theta=0.07$) as a conservative floor.

For each prompt we also compute **cross-topic context load** (“crossload”): the share of accumulated prior-context tokens that belong to threads *other than* the one the prompt is in. Under an ancestor-path-only tree with merge-back condensation, this is the content that would collapse to one-line conclusions — an upper bound on the addressable saving.

Metric	Default ($\theta=0.15$)	Conservative ($\theta=0.07$)
Prompts that switch threads (NEW or RETURN)	32%	19%
RETURN events (re-entry into an earlier thread)	738	—
Sessions with ≥ 2 threads	100%	97%
Sessions interleaved (≥ 1 RETURN)	93%	91%
Mean threads per session	11.0	6.0
Crossload, mean over prompts	51.3%	34.5%
Crossload, token-weighted	44.9%	24.7%

Table 2. Latent thread structure across 68 sessions / 4,434 real prompts (uuid-deduped). The headline structural fact — 93% vs 91% of sessions interleave — is essentially invariant to segmentation strictness, i.e. not an artifact of the threshold. Heuristic crossload spans 25–45% depending on strictness: 10–20 $\times$ the addressable share of Measurement 1 (and itself an underestimate — see §5.1).

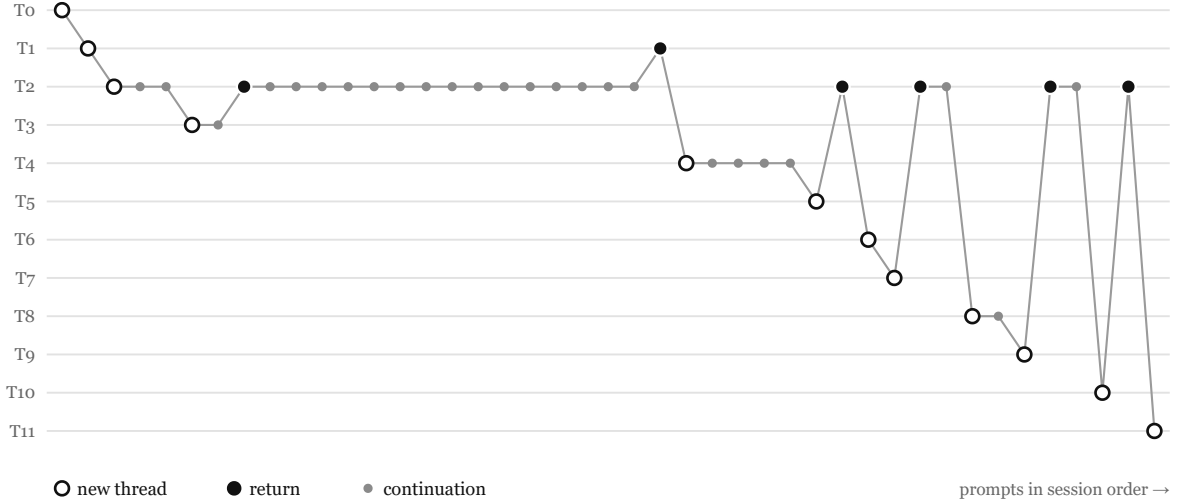


Figure 1. One real session as a braid (43 prompts, 12 threads, $\theta=0.15$, crossload 74%). Each prompt is a dot in its thread’s lane; the path shows session order. Open dots open a new thread; black dots are returns to an earlier thread. Thread T2 — the session’s main experimental work — is re-entered five times, weaving through detours into explanation (T3), a Colab debugging excursion (T4–T6), and publication questions (T7–T10). A manual fork command cannot capture this pattern; users do not branch and come back once, they weave continuously.

Qualitative inspection confirms the segmentation is meaningful rather than noise: in the Figure 1 session the segmenter cleanly separates an experiment-execution thread, a plain-English explanation detour, an environment-debugging excursion, and a cluster of publication-planning questions, with correct return detection each time the user comes back to the experiments. The braid is real, and it is the common case — not an occasional power-user pattern.

5.1 Validation: blind LLM re-segmentation and adjudication

Because lexical segmentation is the weakest link, we validated it on a stratified 10-session sample (5–337 prompts per session): 30 independent blind segmentation passes (three distinct prompting lenses $\times$ 10 sessions, small-model labelers that never saw the heuristic’s labels), 2-of-3 consensus on switch points, and a two-judge adjudication panel over 40 sampled disagreements (85% inter-judge agreement). Three corrections emerged. First, **the heuristic’s switch points are largely wrong**: 29% precision / 39% recall against LLM consensus, and on adjudicated disagreements the

panel sided with the LLM 30–4 (6 unclear); of 26 heuristic-claimed switches the consensus rejected, none survived. An embedding/lexical router (the cheap first rung of the implementation ladder) is therefore not viable — routing requires a small LLM. Second, **the braid is larger than the heuristic claimed**: on the validated segmentations, crossload is **57% mean / 63% token-weighted** (54% excluding one low-agreement session); the heuristic’s 25–45% stands as the conservative floor. Third, a corpus artifact was found and fixed: forked/resumed sessions re-log inherited history (duplicate event uuids); after event-level dedupe the aggregate Phase 1 numbers moved by at most 1.5 percentage points, confirming the structure is not a logging artifact. Under the stricter all-three-passes criterion, interleaving held in 7 of the 10 sample sessions — every session with ≥ 18 prompts except one. The validation also measured a true **cross-dependency rate of $\approx 5\%$** of prompts (prompts in one thread needing another thread’s content) — the share of context that condensation must preserve.

6 Proposal: automatic thread routing with merge-back condensation

The measurements motivate a specific system, targeting exactly the two components CTA leaves open. At each incoming prompt, a lightweight **router** (heuristics plus a small-model classifier) decides: *continuation* of the active thread, *sibling* (new thread branching from the nearest shared ancestor), or *return* to a named earlier thread. The context served for the prompt is its thread’s ancestor path, plus one-line condensed conclusions of closed threads (decisions, constraints, and outcomes mined before condensation — never silently dropped). When a thread completes, a **merge-back condensation** step collapses it to its conclusion in the parent.

Design constraints, learned from the failure modes of context pruning: (a) *misplacement is worse than mis-pruning* — a misrouted prompt hides context the model needed, so the router must be precision-first and fall back to full linear context when confidence is low; (b) *cache economics must be reported* — tree branching naturally reuses the ancestor prefix cache, and merges should be batched with existing compaction moments, but any claimed saving must be stated in cache-adjusted cost, not raw tokens; (c) decisions, corrections, and user preferences are never regenerable and always survive condensation.

Replay ablation (executed, two rounds). At 18 validated RETURN points across 6 sessions — the danger points, where scoped context omits the intervening threads entirely — we replayed the actual next user prompt under different context constructions and had blinded judges score each against the assistant’s actual historical response. Round one (two arms): scoped context with one-line thread stubs is 50.0% smaller; judges preferred FULL 11–5 (2 ties) with a dependency-failure delta of only +1 — suggesting the gap came from condensation losing concrete facts, not from thread isolation itself. Round two tested that directly with a **three-arm component ablation**: FULL vs SUMMARY-only vs SUMMARY+ENTITIES, where the third arm adds an LLM-mined entity ledger (files, commands, decisions, outcomes) from the omitted threads, and all three arms were re-judged blind in shuffled order.

Arm	Context size	Equivalent to actual	Dependency failures	Preferred	Worst
FULL (linear history)	906 KB	10/18	14/18	9	6
SUMMARY-only (stubs)	453 KB (–50%)	9/18	12/18	2	7
SUMMARY+ENTITIES	514 KB (–43%)	9/18	10/18	5	5

Table 3. Three-arm component ablation at 18 return points (blinded, single judge per item, arms shuffled). Entity retention strictly dominates summary-only (preferred 5 vs 2, worst 5 vs 7, dependency failures 10 vs 12) at 7pp extra context, and records the fewest dependency failures of any arm — fewer than full context. Judge-noise calibration: FULL’s equivalence moved 12→10 between the two independent judging rounds over identical answers ($\approx \pm 2$ items), so pairwise differences of that size are within noise.

The component reading: **entity retention is the necessary ingredient of merge-back condensation**. It recovers most of the naive-summary deficit at 43% context reduction, and — strikingly — surfaces cross-thread facts better than the full 906 KB linear history does (10 vs 14 dependency failures), consistent with the buried-context effect that motivates pruning [2,3]. Full context retains a preference edge (9–5) that narrowed from the two-arm round (11–5) and sits near the measured judge-noise band; the strong hypothesis (scoped > full) remains unproven at this power. The full experiment — richer decision-mined condensation, 3-vote judging, cache-adjusted cost — is the next step, with the target now precisely identified: **the ledger, not the prose summary, is what condensation must get right**.

7 Limitations

- **Single-user corpus.** All sessions are one professional user’s; shares will differ across populations. The 93% interleaving rate and 25–45% crossload should be read as an existence proof and feasibility signal, not a population estimate.
- **Segmentation ground truth is itself model-derived.** The §5.1 validation replaces the lexical heuristic with blind LLM consensus plus a 40-item adjudicated sample — much stronger, but still not full human labeling; the LLM passes’ own extra switches were upheld only about half the time when adjudicated, so the validated crossload (54–63%) may carry mild upward bias. The heuristic’s 25–45% is the conservative floor.
- **Ablation power.** $n=18$ return points, one blinded judge per item, and an excerpt-based replay harness (truncated replies, no tool outputs) that handicaps all arms; the measured judge-noise band is $\approx \pm 2$ items, so the preference gaps (9–5, 5–2) are suggestive, not significant, and the entity-arm advantage needs confirmation at higher power.
- **Crossload is an upper bound on savings.** Some cross-thread content (global decisions, shared state) must survive as condensed summaries; realized savings will be lower.
- **Token estimation.** chars/4 estimation of stored content; native usage fields were used only as a cross-check.
- **Quality evidence is pilot-grade only.** The §6 ablation is a floor-setting pilot, not the full experiment; whether well-condensed thread isolation matches or beats linear context remains open, and nothing here presumes that outcome.

8 Conclusion

By intent, real coding-agent context is dominated by action and tool output; the safely evictable explanation layer is $\approx 2\%$ — a negative result that closes one plausible mechanism. By topic, the same context is a braid: 93% of sessions interleave threads, and validated re-segmentation puts the cross-thread share of each prompt’s context at 54–63% (conservative floor 25–45%). A two-round replay ablation at 18 real thread-return moments shows thread-scoping cuts context 43–50%, and its three-arm component analysis isolates the ingredient that matters: an entity ledger mined from the omitted threads strictly beats prose stubs and yields the fewest dependency failures of any arm — fewer than full linear context. The skeleton of the fix is already shipping (manual session forking) and already formalized (CTA); the measurements here narrow the open problem to two

components — an LLM-based placement router (lexical routing was empirically rejected at 29% switch precision) and **entity-preserving condensation on merge-back**, which the component ablation identifies as the place quality is won or lost. Measurement and validation scripts are released with this note.

References

1. P. Hemanth, S. Saha. *Conversation Tree Architecture: A Structured Framework for Context-Aware Multi-Branch LLM Conversations*. arXiv:2603.21278, March 2026.
2. *Less Context, Better Agents: Efficient Context Engineering for Long-Horizon Tool-Using LLM Agents*. arXiv:2606.10209, 2026.
3. *ActiveContext: Escaping the Context Bottleneck via RL Context Curation*. arXiv:2604.11462, 2026.
4. *Branchat: A Tree-Structured Interface for Efficient Revisitation in Long-Horizon LLM Conversations*. CHI 2026 Extended Abstracts, doi:10.1145/3772363.3798792.
5. Anthropic. *Claude Code documentation: Manage sessions* (the `/branch` command and session-tree navigation). <https://code.claude.com/docs/en/sessions>, 2026.
6. Anthropic. *Context editing / context management*. <https://www.anthropic.com/news/context-management>, 2025.
7. C. Packer et al. *MemGPT: Towards LLMs as Operating Systems*. arXiv:2310.08560, 2023.
8. *Conversations in Space: Structuring Non-Linear LLM Interactions on a Canvas*. arXiv:2605.15848, 2026.

Measurement note, Phase 1 (feasibility). Scripts and per-session outputs accompany this note at <https://github.com/mthamil107/sessions-are-braids> (archived: doi:10.5281/zenodo.21486184). Corpus: author’s own Claude Code transcripts; no third-party data. Comments welcome.